

CodeKids

Plataforma Educativa de Programación para Niños

Especificación de arquitectura, diseño de bases de datos, modelo de seguridad en entornos cerrados y manual estructural del sistema.

EQUIPO DE DESARROLLO

Alan / Aquino / Francisco / Alonso

PERIODO DEL PROYECTO

Enero - Agosto 2026

ESTADO DEL DOCUMENTO

Versión 1.0 — Producción Estabilizada

TECNOLOGÍA BASE

Vanilla JS / Tailwind CSS / Firebase Suite

1. Visión General del Proyecto y Filosofía de Diseño

CodeKids es una solución integral en la nube diseñada específicamente para introducir a niños de entre 8 y 14 años en el pensamiento computacional y la lógica de programación. Reconociendo los riesgos inherentes a los entornos web abiertos, el diseño fundamental de CodeKids se rige bajo la doctrina estricta de un **"jardín cerrado" (Closed Garden)**. Esta aproximación garantiza un entorno controlado donde las interacciones externas están completamente mitigadas, permitiendo que la experiencia de aprendizaje ocurra bajo supervisión institucional directa.

1.1 Características Clave del Jardín Cerrado

A diferencia de los sistemas de gestión de aprendizaje (LMS) tradicionales o las redes sociales educativas comunes, CodeKids implementa restricciones estructurales nativas desde su núcleo de software:

- **Inexistencia de Registro Público:** No existe ningún formulario o API expuesta que permita el auto-registro o creación autónoma de credenciales por parte de usuarios externos.
- **Aprovisionamiento Centralizado:** El Administrador del sistema es la única entidad autorizada y capaz de dar de alta cuentas de estudiantes, profesores y directivos dentro del ecosistema de autenticación.
- **Aislamiento de Comunicación Estricto:** Los canales de comunicación y chat bidireccionales están restringidos por grafos de relaciones. Un estudiante solo puede comunicarse con el docente explícitamente asignado a su grupo de clases actual, impidiendo la interacción horizontal entre alumnos o con usuarios externos.

Paradigma de Interfaz de Usuario

La interfaz de la aplicación de escritorio y paneles de control se inspira en el diseño modular de tres columnas popularizado por herramientas de productividad corporativas como Microsoft Teams. Esto permite mantener accesibles de forma simultánea el menú de navegación estructural, la lista de recursos o chats activos y el área principal de trabajo interactivo, minimizando la carga cognitiva en el usuario infantil.

2. Desglose Funcional de Módulos y Roles

El sistema opera mediante una segregación estricta de responsabilidades dividida en tres roles principales definidos en los metadatos de autenticación del usuario.

2.1 Módulo de Estudiantes

Constituye el núcleo del entorno interactivo gamificado. Proporciona las herramientas necesarias para que el alumno asimile los conceptos técnicos mediante experimentación práctica.

- **Visor del Plan de Estudios (Lecciones):** Interfaz interactiva donde los estudiantes consumen materiales didácticos organizados de forma secuencial por niveles de complejidad y cursos específicos.
- **Ecosistema de Entrega de Tareas:** Consola centralizada que detalla los entregables pendientes asignados por el profesor, con contadores de tiempo límite y estados dinámicos (Pendiente, Entregado, Calificado).
- **Laboratorio de Código Independiente:** Un entorno libre (Sandbox) potenciado por programación visual con bloques basados en el motor de Google Blockly. El código generado visualmente puede compilarse y ejecutarse en tiempo real de manera segura.
- **Módulo de Grupos Colaborativos:** Visualización de los equipos de trabajo definidos por el profesor para fomentar el aprendizaje basado en proyectos dirigidos.
- **Canal de Consultas Directas:** Chat interactivo en tiempo real con el docente asignado para la resolución instantánea de dudas.

2.2 Módulo de Profesores

El panel docente proporciona las herramientas de control pedagógico y supervisión analítica del rendimiento de los estudiantes.

- **Gestión de Aulas de Clase (Grupos):** Herramientas para agrupar alumnos, estructurar equipos y modificar asignaciones escolares de manera dinámica.
- **Autoría de Contenido y Tareas:** Módulo para redactar lecciones dinámicas, fijar fechas límite de entregas y programar criterios de evaluación automatizados o manuales.
- **Panel de Analíticas y Progreso:** Integración de tableros gráficos que procesan el avance cuantitativo y cualitativo de cada alumno, permitiendo detectar de forma temprana rezagos en la comprensión lógica.
- **Consola de Atención al Alumno:** Centralizador de chats activos con todos sus alumnos vinculados.

2.3 Módulo de Administradores

Constituye la autoridad suprema del sistema, encargada del soporte técnico estructural y el aprovisionamiento de recursos.

- **Aprovisionamiento Global de Usuarios:** Consola de carga masiva y unitaria para la creación de cuentas asignando roles rígidos en Firestore y contraseñas seguras iniciales.
- **Gestor de Infraestructura Escolar:** Altas, bajas y modificaciones de centros educativos asociados, integrando herramientas de georreferenciación.
- **Bitácora de Auditoría Integral:** Registro persistente de eventos críticos del sistema (accesos, modificaciones de bases de datos, llamadas al backend) para garantizar la trazabilidad de operaciones.

- **Emisor de Difusiones Globales:** Sistema de alertas y notificaciones que se inyectan en tiempo real en las sesiones de todos los usuarios de la plataforma.

3. Especificación Tecnológica del Ecosistema

La arquitectura tecnológica de CodeKids ha sido seleccionada meticulosamente para garantizar un alto rendimiento en navegadores cliente convencionales, eliminando la necesidad de empaquetar software nativo ejecutable.

3.1 Capa de Frontend (Cliente Ligero)

Se optó por un enfoque arquitectónico basado en **JavaScript ES6+ Vanilla**, prescindiendo deliberadamente de frameworks pesados de tipo SPA (como React, Angular o Vue) para maximizar la velocidad de carga en equipos escolares con hardware limitado, reducir las capas de abstracción y agilizar el tiempo de compilación.

Categoría / Librería	Versión	Propósito y Aplicación en CodeKids
JavaScript Nativo	ES6+	Lógica de control principal, manipulación del DOM y consumo de APIs de Firebase.
Tailwind CSS	v3.x	Framework de utilidades CSS para el diseño responsivo del panel de tres columnas.
Google Blockly	v11.2.1	Motor visual para el laboratorio de código mediante abstracción de bloques lógicos.
Leaflet	v1.9.4	Renderizado de mapas interactivos en la Landing Page para la localización geográfica de escuelas inscritas.
Chart.js	v4.4.0	Visualización de métricas de progreso de lecciones y XP en el dashboard de profesores y alumnos.
SweetAlert2	Core Native	Inyección de alertas de interfaz dinámicas y amigables para confirmar acciones lúdicas.
Font Awesome	v6.5.1	Conjunto de iconos vectoriales para la botonera del menú lateral y elementos del dashboard.

3.2 Capa de Backend y Servicios Cloud (Infraestructura Serverless)

El soporte operativo e infraestructura de datos se sustenta sobre la suite de Firebase, permitiendo una escalabilidad elástica sin la sobrecarga administrativa de gestionar servidores tradicionales Linux.

Servicio Cloud	Descripción Funcional e Integración
Firebase Authentication	Manejo seguro del estado de las sesiones de usuario, encriptación de credenciales en reposo y tokens JWT de acceso.
Cloud Firestore	Base de datos documental NoSQL con sincronización por WebSockets para actualizaciones en tiempo real del chat y tablas de posiciones.
Firebase Storage	Almacenamiento de objetos multimedia binarios para recursos de lecciones, capturas de tareas entregadas e imágenes de perfil de los avatares.
Firebase Functions	Entorno serverless en Node.js encargado de ejecutar tareas privilegiadas (creación de usuarios, auditoría avanzada y proxy de IA).
Firebase Hosting	Red de entrega de contenido (CDN) global y optimizada que sirve los archivos estáticos HTML, CSS y JS de la aplicación de manera segura (HTTPS).
Render.com	Infraestructura complementaria de contingencia orquestada mediante un manifiesto <code>render.yaml</code> para el despliegue automático.

4. Arquitectura del Repositorio de Código

La organización de los archivos fuente dentro del repositorio `CodeKids-Clean/` sigue un patrón limpio y modular enfocado en la separación clara de vistas públicas y privadas.

```

CodeKids-Clean/
|
|— index.html           # Enrutador principal de entrada (Redirección inicial)
|— firebase.json        # Configuración de puertos de emulación y reglas de
hosting
|— firestore.rules      # Reglas declarativas de control de acceso para la base
de datos
|— firestore.indexes.json # Definición de índices compuestos obligatorios para
Firestore
|— storage.rules        # Políticas de seguridad aplicadas a archivos multimedia
subidos
|— render.yaml          # Configuración de microservicios e infraestructura en
Render
|— package.json         # Gestión de dependencias npm, scripts de despliegue y
herramientas
|
|— Vistas_Publicas/     # Páginas de acceso libre sin token de autenticación
|   |— Pagina_Inicio.html # Landing page informativa con el mapa interactivo de
escuelas
|   |— Inicio_De_Sesion.html # Portal único de autenticación de credenciales de
usuario
|   |— Beneficios_Profesor.html # Documentación comercial para la adopción de centros
escolares
|
|— auth/
|   |— register.html    # Interfaz de registro manual (Protección restrictiva:
solo administradores)
|
|— Dashboard/          # Módulos privados restringidos por Token JWT y Roles
|   |— Dashboard_Administrador/ # Panel de control de infraestructura, escuelas y
auditorías globales
|   |— Dashboard_Estudiante/   # Interfaz lúdica: acceso a cursos, tareas y espacio
Blockly
|   |— Dashboard_Profesor/     # Consola de seguimiento escolar, calificaciones y chat
docente
|
|— js/
|   |— firebase-config.js      # Lógica modular pura de la aplicación cliente
autenticados
|   |— auth.js                # Inicialización y exposición de SDKs de Firebase
|   |— interceptor de rutas   # Manejador de estado de sesión, cambio de claves e
|   |— app.js                 # Orquestador del ciclo de vida general y renderizado del
DOM principal
|   |— gamification.js        # Algoritmos de cálculo de XP, subida de niveles
automáticos e insignias
|   |— laboratorio.js         # Inyección del espacio Blockly y compilación del árbol
sintáctico (AST)
|   |— admin.js               # Control de peticiones masivas y envío de logs de
auditoría
|   |— chat-enhancements.js   # Manejo de sockets de mensajería y persistencia local de
hilos
|   |— state-manager.js       # Implementación de almacén de estado global in-memory de
la app

```



```
|  
├─ css/                # Estilos globales y compilación final de utilidades  
Tailwind CSS  
├─ images/            # Activos visuales estáticos, sprites de medallas y  
marcos de avatar  
├─ functions/         # Código fuente de las funciones Cloud en Node.js  
(Servicios Serverless)  
├─ scripts/           # Utilidades de mantenimiento CLI para limpiar datos de  
prueba en caliente  
├─ AVISOS PARA EL ASESOR/  
equipos internos      # Repositorio de requerimientos y sincronización de  
└─ CARPETA INFORMATIVA/  
documentación legal   # Manuales de usuario finales, material didáctico y
```

5. Arquitectura y Esquema de Base de Datos (Cloud Firestore)

Firestore organiza los datos del ecosistema CodeKids de forma jerárquica a través de colecciones raíz y documentos estructurados en formato NoSQL orientado a objetos. A continuación, se detalla la composición estructural de cada colección:

5.1 Colección Raíz: `/users/{userId}`

Almacena el perfil maestro de cada usuario del sistema. El identificador del documento `userId` mapea directamente al identificador único provisto por Firebase Authentication.

uid: String — Identificador global único de seguridad.

name: String — Nombre completo del usuario.

email: String — Correo institucional.

role: String — Discriminador estricto de permisos: `"admin"`, `"teacher"`, o `"student"`.

schoolId: String — Puntero de relación con la escuela asociada.

isFirstLogin: Boolean — Flag indicador para obligar al cambio de contraseña.

createdAt: Timestamp — Fecha y hora de alta en el sistema.

gamification: Map *(Solo presente si el rol es estudiante)*

— **currentXP:** Number — Puntos de experiencia totales acumulados.

— **level:** Number — Nivel matemático actual calculado del **1** al **10**.

— **avatarFrameId:** String — Cosmético estético seleccionado para el perfil.

Subcolecciones de Usuario:

- `/users/{userId}/notifications` : Historial individual de alertas, avisos e invitaciones a grupos.
- `/users/{userId}/progress` : Registro puntual de lecciones consumidas, guardando el id de la lección, la fecha de compleción y la puntuación obtenida en su respectivo cuestionario.
- `/users/{userId}/earnedBadges` : Subcolección conteniendo las insignias desbloqueadas por hitos de programación alcanzados.

5.2 Colección Raíz: `/groups/{groupId}`

Define las fronteras lógicas de las aulas virtuales creadas por el cuerpo docente.

name: String — Nombre descriptivo del aula (ej. "Lógica A - Primaria").

teacherId: String — Relación foránea con el documento del usuario docente creador.

schoolId: String — Enlace jerárquico del plantel educativo.

studentsList: Array [String] — Arreglo ordenado conteniendo las UIDs de los estudiantes inscritos.

updatedAt: Timestamp — Control de auditoría de modificaciones temporales.

5.3 Colección Raíz: `/lessons/{lessonId}` y `/tasks/{taskId}`

Administración de contenidos educativos y entregables escolares con fechas límite embebidas.

title: String — Título de la lección o tarea.

content: String — Cuerpo de la lección estructurado en Markdown o HTML seguro sanitizado.

difficulty: String — Niveles categorizados (`"Principiante"` , `"Intermedio"` , `"Avanzado"`).

dueDate: Timestamp (*Solo para tareas*) — Fecha de corte absoluto tras la cual el sistema bloquea entregas.

xpReward: Number — Puntos otorgados automáticamente por completar satisfactoriamente la actividad.

5.4 Colección Raíz: `/chats/{chatId}`

Controla los canales de comunicación. El `chatId` compuesto sigue una regla estricta: `teacherId_studentId` para evitar duplicidad de canales y asegurar que ninguna tercera parte intervenga. Contiene una subcolección interna denominada `/messages` que ordena de forma cronológica cada mensaje enviado con su emisor, texto y marca temporal exacta.

5.5 Colección Raíz: `/schools/{schoolId}`

Catálogo central de las sedes educativas afiliadas a la plataforma CodeKids.

name: String — Nombre oficial de la escuela.

address: String — Dirección física completa del inmueble.

coordinates: GeoPoint — Par ordenado compuesto por latitud y longitud (*lat, lng*) requerido nativamente por Leaflet para posicionar el marcador gráfico en el mapa público.

6. Modelo de Seguridad Avanzado y Flujos Críticos

6.1 Ciclo de Vida de Autenticación de Nuevos Usuarios

Para garantizar el mantenimiento estricto de la política de jardín cerrado, el proceso de inicio y ciclo de vida de credenciales cumple de manera lineal el siguiente algoritmo del sistema:

1. Un usuario con rol administrador accede a la ruta privada `/auth/register.html` e ingresa los datos del nuevo estudiante o profesor.
2. Una Firebase Function intercepta la petición, valida la firma del token del administrador y realiza la inserción de credenciales usando el SDK de administración de Firebase, generando una clave temporal aleatoria.
3. Al realizar el primer inicio de sesión en `Inicio_De_Sesion.html`, el script `auth.js` detecta que el campo documentado `isFirstLogin == true`.
4. El sistema interrumpe inmediatamente el flujo de enrutamiento al Dashboard y despliega de manera forzada un modal restrictivo que obliga al usuario a parametrizar una nueva contraseña alfanumérica personal y segura.
5. Una vez confirmada la actualización de la contraseña, el flag cambia a `false` en Firestore, liberando el paso hacia el Dashboard correspondiente mapeado según su rol institucional.

Reglas de Seguridad Estrictas en Cloud Firestore

Las políticas declarativas implementadas en `firestore.rules` impiden cualquier inyección maliciosa de datos. Las reglas aseguran que un usuario con rol de Estudiante solo cuente con permisos de lectura sobre su propio documento en `/users/{auth.uid}`, prohibiendo modificaciones manuales directas sobre campos sensibles como su saldo de puntos de experiencia (XP) o nivel, los cuales solo pueden ser incrementados mediante transacciones seguras de Firebase Functions del lado del servidor tras validar hitos legítimos de Blockly.

7. Cody: Asistente Educativo de Inteligencia Artificial Integrado

La plataforma incluye un tutor interactivo virtual inteligente llamado **Cody**, disponible dentro del dashboard del estudiante para resolver dudas técnicas sobre conceptos algorítmicos.

7.1 Mecánica de Integración y Proxy de Seguridad

Para evitar exponer directamente endpoints o incurrir en costos elevados de APIs de terceros comerciales, el sistema implementa una infraestructura local híbrida:

- **Motor de Inferencia:** Se utiliza un modelo fundacional de lenguaje optimizado y ejecutado localmente mediante la herramienta **Ollama** en servidores dedicados.

- **Capa Intermedia (Firebase Functions Proxy):** La aplicación frontend nunca realiza llamadas directas al servidor de Ollama. Las peticiones viajan cifradas hacia una Cloud Function que actúa como pasarela segura de backend.
- **Políticas de Control de Consumo (Rate Limiting):** La Cloud Function extrae la dirección IP de origen de la llamada de red y valida contra un contador temporal en caché. El sistema impone un límite estricto de **5 solicitudes por hora por IP** para prevenir ataques de denegación de servicio (DoS) y el agotamiento de recursos del servidor de inferencia.
- **Inyección de Contexto Educativo (Prompt Guardrails):** Antes de enviar la pregunta del menor al modelo de lenguaje de Ollama, la función antepone un system prompt estricto en español que fuerza a Cody a responder usando metáforas amigables, lenguaje apto para niños y restringiendo cualquier temática ajena al ámbito de la programación escolar.

8. Entorno de Desarrollo, Emulación y Despliegue

El desarrollo y prueba de software local está respaldado por el uso intensivo de **Firebase Emulator Suite**, lo que garantiza que los desarrolladores puedan realizar pruebas de integración y simular escenarios de producción de manera aislada sin alterar la base de datos viva ni incurrir en consumos de red innecesarios.

Las direcciones y puertos configurados en el archivo maestro de red local son:

Servicio de Emulación	Endpoint Local de Desarrollo
Authentication Emulator	<code>http://127.0.0.1:9099</code>
Cloud Firestore Emulator	<code>http://127.0.0.1:8082</code>
Cloud Functions Emulator	<code>http://127.0.0.1:5001</code>
Firebase Hosting Emulator	<code>http://127.0.0.1:5002</code>
Emulator Suite UI (Consola Visual)	<code>http://127.0.0.1:4001</code>

8.1 Pipelines de Despliegue en Producción

Una vez que los cambios superan la etapa de aseguramiento de calidad en el entorno emulado, el despliegue del software se ejecuta mediante dos alternativas redundantes parametrizadas:

- **Canal Principal (Firebase Hosting CLI):** La ejecución del comando formal `firebase deploy` realiza el empaquetado, compresión optimizada y distribución global de todos los componentes estáticos y funciones serverless del proyecto hacia los servidores de producción de Google.
- **Canal Secundario (Render.com Git-Ops Pipeline):** El archivo de manifiesto de infraestructura como código `render.yaml` permite enlazar el repositorio de Git con la plataforma Render,

automatizando un webhook que vuelve a compilar y servir el entorno estático ante cualquier actualización o caída imprevista en el proveedor primario.